Computer Programming 2
(Intermediate Programming)

Nomer B. Aguado

Bachelor of Science in Computer Science
Bachelor of Science in Information Technology
College of Computing Studies

Modified January 2025

Table of Contents

Module 5: ARRAYS
Introduction 77
Learning Outcomes 77
Lesson 1. What is an Array? 78
Lesson 2. One-Dimensional Array 79
Lesson 3. Accessing Elements of an Array 80
Lesson 4. Initializing Array 85
Lesson 5. One-Dimensional Array with Functions 89
Lesson 6. Two-Dimensional Array 92
Lesson 7. Accessing Elements of a Two-Dimensional Array 95
Lesson 8. Initializing Two-Dimensional Array 97
Lesson 9. Multidimensional Arrays 101
Assessment Task 104
Summary 105

REFERENCES 106

Module 6: PASSING ARRAYS TO A FUNCTION
Introduction 107
Learning Outcomes 107
Lesson 1. Passing 1-D Array to a Function 108
Lesson 2. Passing 2-D Array to a Function 110
Assessment Task 114
Summary 114
REFERENCES 114

Module 7: STRINGS
Introduction 115
Learning Outcomes 115
Lesson 1. What is Strings? 116
Lesson 2. String Literal 116
Lesson 3. printf() and scanf() revisited 118
Lesson 4. C Programming Strings 119
Lesson 5. String Manipulation using Library Functions 123
Lesson 5.1 C strlen() Function 124
Lesson 5.2 C strcpy() Function 125
Lesson 5.3 C strcat() Function 126
Lesson 5.4 C strcmp() Function 128
Lesson 5.5 C strlwr() Function 130
Lesson 5.6 C strupr() Function 131
Assessment Task 134
Summary 135
REFERENCES 135

MODULE 5

ARRAYS

Introduction

An array is a group (or collection) of same data types. For example, an **int** array holds the elements of **int** types while a **float** array holds the elements of **float** types. Why do we need arrays in programming? Consider a scenario where you need to find out the average of 100 integer numbers entered by user. In C, you have two ways to do this: 1. Define 100 variables with **int** data type and then perform 100 **scanf()** operations to store the entered values in the variables and then at last calculate the average of them. 2. Have a single integer array to store all the values, loop the array to store all the entered values in array and later calculate the average. Obviously the second solution, it is convenient to store same data types in one single variable and later access them using array index (Singh, n.d.).

In this module, we are going to discuss and understand the use of arrays and it's different types.

Learning Outcomes

At the end of this module, students should be able to:

- Discuss and explain what is an array in data structures.
- Apply the use 1-Dimensional, 2-Dimensional and 3-Dimensional arrays.

Lesson 1. What is an Array? (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), An array is a collection of one or more values of the same type. Each value is called an **element** of the array. The elements of the array share the same variable name but each element has its own unique index number (also known as a subscript). An array can be of any type, For example: **int**, **float**, **char** etc.

Note: If an array is of type int then it's elements must be of type int only.

For example, to store roll no. of 100 students, we have to declare an array of size 100 i.e **roll_no[100]**. Here, size of the array is 100, so it is capable of storing 100 values. In C, index or subscript starts from 0, so roll_no[0] is the first element, roll_no[1] is the second element and so on.

Note: The last element of the array will be at roll_no[99] not at roll_no[100] because the index starts at 0.

Figure 5.7 Structure of an Array (Overiq.com, 2020)

Arrays can be **single** or **multidimensional**. The number of subscript or index determines the dimensions of the array. An array of one dimension is known as a **one-dimensional array**

or 1-D array, while an array of two dimensions is known as a two-dimensional array or 2-D array (We're going to talk about 2-D array in later texts).

Let's start with a one-dimensional array.

Lesson 2. One-Dimensional Array (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), conceptually you can think of a one-dimensional array as a row, where elements are stored one after another.

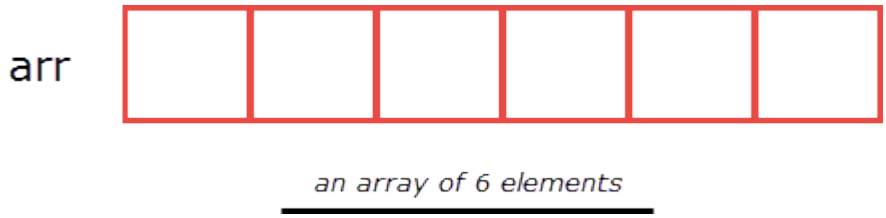

Figure 5.8 One Dimensional Array (Overiq.com, 2020)

Syntax:

```
datatype array_name[size];
```

- **Datatype -** It denotes the type of the elements in the array.
- **Array_name -** Name of the array. It must be a valid identifier. You can follow the rules of naming a variable.
- **Size -** Number of elements an array can hold. For example:

```
int num[100];
float temp[20];
char ch[50];
```

Interpretation:

- **num** is an array of type **int**, which can only store **100 elements** of type **int**.
- **temp** is an array of type **float**, which can only store **20 elements** of type **float**.
- **ch** is an array of type **char**, which can only store **50 elements** of type **char**.

Note: When an array is declared it contains garbage values. For example,

The individual or garbage values/elements in the array:

```
num[0], num[1], num[2], ....., num[99]
temp[0], temp[1], temp[2], ....., temp[19]
ch[0], ch[1], ch[2], ....., ch[49]
```

Lesson 3. Accessing Elements of an Array (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), the elements of an array can be accessed by specifying array name followed by subscript or index inside square brackets (i.e **[ ]**). Array subscript or index starts at 0. If the size of an array is 10 then the first element is at index 0, while the last element is at index 9. The first valid subscript (i.e 0) is known as the lower bound, while last valid subscript is known as the upper bound.

Example:

```
int my_arr[5];
```

The elements of this array are;

- First element – **my_arr[0]**
- Second element – **my_arr[1]**
- Third element **– my_arr[2]**
- Fourth element – **my_arr[3]**
- Fifth element – **my_arr[4]**

Note: Access Elements out of its bound.

Suppose you declared an array of 10 elements. Let's say,

```
Int testArray[10];
```

You can access the array elements from indices **testArray[0]** to **testArray[9].**

Now let's say if you try to access **testArray[12].** The element is not available. This may cause unexpected output (undefined behavior). Sometimes you might get an error and some other time your program may run correctly (Programiz.com, n.d.).

Example 1:

The following program uses for loop to take input and print elements of a 1-D array.

```
#include<stdio.h>

int main()
{
    int arr[5], i;

    for(i = 0; i < 5; i++)
    {
        printf("Enter a[%d]: ", i);
        scanf("%d", &arr[i]);
    }

    printf("\nPrinting elements of the array: \n\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
Enter a[0]: 11
Enter a[1]: 22
Enter a[2]: 34
Enter a[3]: 4
Enter a[4]: 34

Printing elements of the array:

11 22 34 4 34
```

How it works?

In Line 5, we have declared an array of 5 integers and variable i of type int. Then a for loop is used to enter five elements into an array. In scanf() we have used & operator (also known as the address of operator) on element arr[i] of an array, just like we had done with variables of type int, float, char etc. Line 13 prints "Printing elements of the array" to the console. The second for loop prints all the elements of an array one by one (Overiq.com, 2020).

Example 2: Sum of elements of an Array

```
#include<stdio.h>

int main()
{
    int arr[5], i, s = 0;

    for(i = 0; i < 5; i++)
    {
        printf("Enter a[%d]: ", i);
        scanf("%d", &arr[i]);
    }

    for(i = 0; i < 5; i++)
    {
        s += arr[i];
    }

    printf("\nSum of elements = %d ", s);

    return 0;
}
```

Output:

```
Enter a[0]: 22
Enter a[1]: 33
Enter a[2]: 56
Enter a[3]: 73
Enter a[4]: 23

Sum of elements = 207
```

How it works?

The first for loop asks the user to enter five elements into the array. The second for loop reads all the elements of an array one by one and accumulate the sum of all the elements in the variable **s**. Note that it is necessary to initialize the variable **s** to 0, otherwise, we will get the wrong answer because of the garbage value of **s** (Overiq.com, 2020).

Lesson 4. Initializing Array (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), when an array is declared inside a function the elements of the array have garbage value. If an array is **global** or **static**, then its elements are automatically initialized to 0. We can explicitly initialize elements of an array at the time of declaration using the following syntax:

Syntax:

```
datatype array_name[size] = { val1, val2, val3, ..... valN };
```

- **Datatype –** is the type of elements of an array.
- **Array_name –** is the variable name, which must be any valid identifier.
- **Size –** is the size of the array.
- **val1, val2 … -** are the constants known as initializers. Each value is separated by a comma (,) and then there is a semi-colon (;) after the closing curly brace (}).

Example:

```
float temp[5] = {12.3, 4.1, 3.8, 9.5, 4.5}; // an array of 5
floats

int arr[9] = {11, 22, 33, 44, 55, 66, 77, 88, 99}; // an array of
9 integers
```

While initializing 1-D array it is optional to specify the size of the array, so you can also write the above statements as:

```
float temp[] = {12.3, 4.1, 3.8, 9.5, 4.5}; // an array of 5
floats

int arr[] = {11, 22, 33, 44, 55, 66, 77, 88, 99}; // an array of
9 integers
```

Note: If the number of initializers is less than the specified size then the remaining elements of the array are assigned a value of 0.

```
float temp[5] = {12.3, 4.1};
```

The size of **temp** array is **5** but there are only two initializers. After this initialization the elements of the array are as follows:

`temp[0]` is `12.3`
`temp[1]` is `4.1`
`temp[2]` is `0`
`temp[3]` is `0`
`temp[4]` is `0`

Change Value of Array Elements

You can change the value of the array elements by initializing the ArrayName, Index and new value (Programiz.com, n.d.). **For example:**

```
int mark[5] = {19, 10, 8, 17, 9}

// make the value of the third element to -1
mark[2] = -1;

// make the value of the fifth element to 0
mark[4] = 0;
```

The updated value of elements:

```
int mark[5] = {19, 10, -1, 17, 0}
```

Example 1: Arrays with Input/Output

```
#include <stdio.h>

int main() {
  int values[5];

  printf("Enter 5 integers: ");

  // taking input and storing it in an array
  for(int i = 0; i < 5; ++i) {
     scanf("%d", &values[i]);
  }

  printf("Displaying integers: ");

  // printing elements of an array
  for(int i = 0; i < 5; ++i) {
     printf("%d\n", values[i]);
  }

  return 0;
}
```

Output:

```
Enter 5 integers:
1
2
3
4
5

Displaying integers:
1
2
3
4
5
```

Here, we have used a **for loop** to **take 5 inputs** from the user and **store them in an array**. Then, using **another for loop** to **display** the elements stored in the array (Programiz.com, n.d.).

Example 2: Calculate Average

```
#include <stdio.h>
int main()
{
     int marks[10], i, n, sum = 0, average;

     printf("Enter number of elements: ");
     scanf("%d", &n);

     for(i=0; i<n; ++i)
     {
          printf("Enter number%d: ",i+1);
          scanf("%d", &marks[i]);

       // adding integers entered by the user to the sum variable
          sum += marks[i];
     }

     average = sum/n;
     printf("Average = %d", average);

     return 0;
}
```

Let's say, we will **compute the average of 5 numbers or elements.**

Output:

```
Enter number of elements: 5

Enter Number 1: 45
Enter Number 2: 35
Enter Number 3: 38
Enter Number 4: 31
Enter Number 5: 49

Average = 39
```

Lesson 5. One Dimensional Array with Functions (Overiq.com, 2020)

Based in the tutorial “C Programming 101” by Overiq.com (2020), you can pass elements of 1-D array just like any normal variables.

Example 1:

```
#include<stdio.h>

int main()
{
    int my_arr[] = {13,56,71,38,93}, i;

    for(i = 0; i < 5; i++)
    {
        // passing one element at a time to odd_or_even()
function
        odd_or_even(my_arr[i]);
    }

    return 0;
}

void odd_or_even(int a) //a function
{
    if(a % 2 == 0)
    {
        printf("%d is even\n", a);
    }

    else
    {
        printf("%d is odd\n", a);
    }
}
```

Output:

```
13 is odd
56 is even
71 is odd
38 is even
93 is odd
```

Passing the whole Array to a Function

Based in the tutorial “C Programming 101” by Overiq.com (2020), just like normal variables you can pass an array variable to a function. But before you do so, make sure the formal arguments are declared as an array variable of same data type. For example:

```
int main()
{
    ...
    int a[10];
    ...
    function_1(a);
    ...
    return 0;
}

void function_1(int arr[10])
{
    ...
    Statement 1;
    ...
}
```

Here, we are passing an **array of 10 integers** to **function_1()**, that's why the formal argument of **function_1()** is also declared as an **array of 10 integers**.

It is optional to specify the size of the array in the formal arguments. This means you can also declare formal argument of **function_1()** as follows (Overiq.com, 2020):

```
void function_1(int arr[])
{
    ...
    statement 1;
    ...
}
```

While learning about formal and actual arguments, we have learned that changes made in the formal arguments do not affect the actual arguments. This is not the case with arrays. When an array is passed as an actual argument, the function gets access to the original array, so any changes made inside the function will affect the original array. For example:

```
#include<stdio.h>
void new_array(int a[]);

int main()
{
    int my_arr[] = {13,56,71,38,93}, i;

    printf("Original array: \n\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", my_arr[i]);
    }

    new_array(my_arr);

    printf("\n\nModified array : \n\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", my_arr[i]);
    }

    return 0;
}

void new_array(int a[])
{
    int i;

    // multiply original elements by 2

    for(i = 0; i < 5; i++)
    {
        a[i] = 2 * a[i];
    }
}
```

Output:

```
Original Array:

13 56 71 38 93

Modified Array:

26 112 142 76 186
```

How it works?

The first for loop in **main()** function prints the initial values of the elements of an array. In line 15, **new_array()** function is called with an actual argument of **my_arr.** The Control is transferred to function **new_array().** The function multiplies each element of the array by 2 and assigns back this new value to the current index. Since **new_array()** is working on the original array, not on a copy of the original array, any changes made by **new_array()** function affect the original array. When the function finishes, control again passes back to **main()** function, where second for loop prints the elements of the array (Overiq.com, 2020).

Lesson 6. Two-Dimensional Array (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), the syntax declaration of 2-D array is not much different from 1-D array. In 2-D array, to declare and access elements of a 2-D array we use **2 subscripts** instead of 1.

Syntax:

```
datatype array_name[ROW][COL];
```

The total number of elements in a 2-D array is **ROW (Row) *COL (Column).** Let's take an example.

```
int arr[2][3];
```

This array can store **2*3 = 6 elements**. You can visualize this 2-D array as a matrix of **2 rows** and **3 columns.**

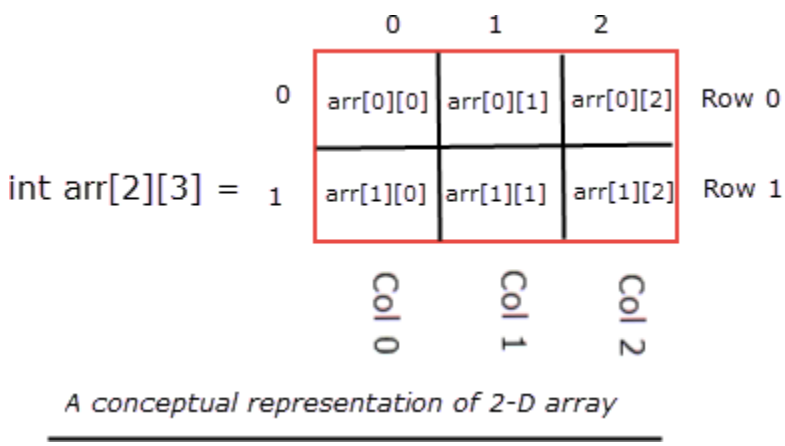

Figure 5.9. Two-Dimensional Array (Overiq.com, 2020)

The individual elements of the above array can be accessed by using two subscripts instead of one. The first subscript denotes row number and second denotes column number. As we can see in the above image both rows and columns are indexed from 0. So the first element of this array is at arr[0][0] and the last element is at arr[1][2].

- **arr[0][0]** - refers to the first element
- **arr[0][1]** - refers to the second element
- **arr[0][2]** - refers to the third element
- **arr[1][0]** - refers to the fourth element
- **arr[1][1]** - refers to the fifth element
- **arr[1][2]** - refers to the sixth element

If you try to access an element beyond valid **ROW** and **COL**, C compiler will not display any kind of error message, instead, a garbage value will be printed. It is the responsibility of the programmer to handle the bounds (Overiq.com, 2020).

- **arr[1][3]** - a garbage value will be printed, because the last valid index of COL is 2
- **arr[2][3]** - a garbage value will be printed, because the last valid index of ROW and COL is 1 and 2 respectively

Just like 1-D arrays, we can only also use constants and symbolic constants to specify the size of a 2-D array.

```
#define ROW 2
#define COL 3

int i = 4, j = 6;
int arr[ROW][COL]; // OK
int new_arr[i][j]; // ERROR
```

Lesson 7. Accessing Elements of a Two-Dimensional Array
(Overiq.com, 2020)

Based in the tutorial “C Programming 101” by Overiq.com (2020), to process elements of a 2-D array, we use two nested loops. The outer for loop to loop through all the rows and inner for loop to loop through all the columns.

Example:

```
#include<stdio.h>
#define ROW 3
#define COL 4

int main()
{
    int arr[ROW][COL], i, j;

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            printf("Enter arr[%d][%d]: ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    printf("\nEntered 2-D array is: \n\n");

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            printf("%3d ", arr[i][j] );
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter arr[0][0]: 11
Enter arr[0][1]: 35
Enter arr[0][2]: 73
Enter arr[0][3]: 831
Enter arr[1][0]: 3
Enter arr[1][1]: 40
Enter arr[1][2]: 31
Enter arr[1][3]: 93
Enter arr[2][0]: 35
Enter arr[2][1]: 10
Enter arr[2][2]: 52
Enter arr[2][3]: 81

Entered 2-D array is:

11 35 73 831
3 40 31 93
35 10 52 81
```

Lesson 8. Initializing Two-Dimensional Array (Overiq.com, 2020)

Initialization of 2-D array is similar to a 1-D array (Overiq.com, 2020). For Example:

```
int temp[2][3] = {
    { 1, 2, 3 }, // row 0
    {11, 22, 33} // row 1
};
```

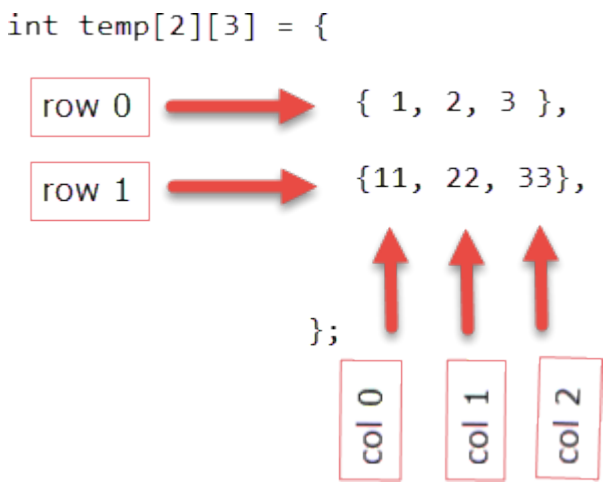

Figure 5.10 Initializing a 2-D Array (Overiq.com, 2020)

After this initialization, each element is as follows:

```
temp[0][0] : 1
temp[0][1] : 2
temp[0][2] : 3
temp[1][0] : 11
temp[1][1] : 22
temp[1][2] : 33
```

This is another way of initializing 2-D Array:

```
int my_arr[4][3] = {
                        {10},
                        {77, 92},
                        {33, 89, 44},
                        {12, 11}
                    };
```

The size of **my_arr is 4*3=12** , but in the initialization, we have only specified the value of 8 elements. In such cases, the remaining elements will be given the value of 0.

The individual elements are as follows:

```
my_arr[0][0] : 10
my_arr[0][1] : 0
my_arr[0][2] : 0

my_arr[1][0] : 77
my_arr[1][1] : 92
my_arr[1][2] : 0

my_arr[2][0] : 33
my_arr[2][1] : 89
my_arr[2][2] : 44

my_arr[3][0] : 12
my_arr[3][1] : 11
my_arr[4][2] : 0
```

In 2-D arrays, it is optional to specify the first dimension but the second dimension must always be present. This works only when you are declaring and initializing the array at the same time (Overiq.com, 2020). For example:

```
int two_d[][3] = {
                    {13,23,34},
                    {15,27,35}
                };
```

As discussed earlier you can visualize a 2-D array as a matrix.

Example: Addition of 2 matrices

```
#include<stdio.h>
#define ROW 2
#define COL 3

int main()
{
    int mat1[ROW][COL], mat2[ROW][COL], mat3[ROW][COL];
    int i, j;

    printf("Enter first matrix: \n\n");

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            printf("Enter a[%d][%d]: ", i, j);
            scanf("%d", &mat1[i][j]);
        }
    }

    printf("\nEnter Second matrix: \n\n");

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            printf("Enter a[%d][%d]: ", i, j);
            scanf("%d", &mat2[i][j]);
        }
    }

    // add mat1 and mat2

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
```

```
        {
            mat3[i][j] = mat1[i][j] + mat2[i][j] ;
        }
    }

    printf("\nResultant array: \n\n");

    // print resultant array

    for(i = 0; i < ROW; i++)
    {
        for(j = 0; j < COL; j++)
        {
            printf("%5d ", mat3[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter first matrix:

Enter a[0][0]: 12
Enter a[0][1]: 32
Enter a[0][2]: 13
Enter a[1][0]: 35
Enter a[1][1]: 54
Enter a[1][2]: 35

Enter Second matrix:

Enter a[0][0]: 57
Enter a[0][1]: 64
Enter a[0][2]: 58
Enter a[1][0]: 72
Enter a[1][1]: 84
Enter a[1][2]: 29

Resultant array:

mat1 + mat2 =

69 96 71
107 138 64
```

How it works?

Two matrices can be added or subtracted, only if they have the same dimension. In other words, a matrix of size **2*3** can be added to another matrix of **2*3**, but you can't add or subtract it to a matrix of **2*4** or **3*2**. The resultant array will be a matrix of the same dimension as the original two. First two for loops asks the user to enter two matrices. The third for loop adds corresponding elements of **mat1** and **mat2** in a new array **mat3**. Fourth for loop prints the elements of array **mat3**.

Lesson 9. Multidimensional Arrays (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), You can even create an array of 3 or more dimensions or more, but generally, you will never need to do so unless you go deeper in data structures. Therefore, we will restrict ourself to 3-D arrays only.

Here is how you can declare an array of 3 dimensions.

```
int arr[2][3][2];
```

3-D array uses three indexes or subscript. This array can store **2*3*2=12 elements**.

Here is how to initialize a 3-D Array.

```
int three_d[2][3][4] = {
                            {
                                {12,34,56,12},
                                {57,44,62,14},
                                {64,36,91,16},
                            },

                            {
                                {87,11,42,82},
                                {93,44,12,99},
                                {96,34,33,26},
                            }
                        };
```

You can think of this array as 2 2-D arrays and each of these 2-D array has 3 rows and 4 columns;

Here are the individual elements of the array:

First Row:

```
three_d[0][0][0] :  12
three_d[0][0][1] :  34
three_d[0][0][2] :  56

three_d[0][0][3] :  12
three_d[0][1][0] :  57
three_d[0][1][1] :  44

three_d[0][1][2] :  62
three_d[0][1][3] :  14
three_d[0][2][0] :  64

three_d[0][2][1] :  36
three_d[0][2][2] :  91
three_d[0][2][3] :  16
```

Second Row:

```
three_d[1][0][0] :  87
three_d[1][0][1] :  11
three_d[1][0][2] :  42

three_d[1][0][3] :  82
three_d[1][1][0] :  93
three_d[1][1][1] :  44

three_d[1][1][2] :  12
three_d[1][1][3] :  99
three_d[1][2][0] :  96

three_d[1][2][1] :  34
three_d[1][2][2] :  33
three_d[1][2][3] :  26
```

You can watch this video tutorial by Neso Academy in using C Arrays:

- *Definition of Array: https://www.youtube.com/watch?v=55l-aZ7_F24&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP*
- *Declaration of Array: https://www.youtube.com/watch?v=Bqud0_ozgcc&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP&index=2*
- *Accessing Array Elements: https://www.youtube.com/watch?v=g0ClJ28-8LE&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP&index=3*
- *Initializing an Array: https://www.youtube.com/watch?v=GtcRReso9Xk&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP&index=4*
- *Introduction to 2-D Arrays - https://www.youtube.com/watch?v=J1aQ9JN4vZY&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP&index=10*
- *Introduction to 3-D Arrays - https://www.youtube.com/watch?v=bbkdiUbou74&list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP&index=11*

Assessment Task

Coding Problems: Answer the ff. programs based on the given assessment format.

1. Create a C Program that will take a series of 10 inputs stored in an Array and calculate its sum.

   Sample Output:

```
Input the number of elements to be stored in the array :10
Input 10 elements in the array :
element - 0 : 2
element - 1 : 5
element - 2 : 4
element - 3 : 6
element - 4 : 10
element - 5 : 12
element - 6 : 65
element - 7 : 51
element - 8 : 22
element - 9 : 10
Sum of all elements stored in the array is : 187
```

2. Create a C Program with an array size 4 * 4 and print the matrix.

   Sample Output:

```
Input elements in the matrix :
element - [0],[0] : 2
element - [0],[1] : 5
element - [0],[2] : 65
element - [0],[3] : 8
element - [1],[0] : 45
element - [1],[1] : 6
element - [1],[2] : 5
element - [1],[3] : 2
element - [2],[0] : 3
element - [2],[1] : 1
element - [2],[2] : 2
element - [2],[3] : 5
element - [3],[0] : 6
element - [3],[1] : 8
element - [3],[2] : 4
element - [3],[3] : 5

The matrix is :

2       5       65      8
45      6       5       2
3       1       2       5
6       8       4       5


--------------------------------
```

3. Create a C Program that will calculate the sum of Two (2) Matrices.

Sample Output:

```
-----------------------------
Input the size of the square matrix: 2
Input elements in the first matrix :
element - [0],[0] : 11
element - [0],[1] : 43
element - [1],[0] : 24
element - [1],[1] : 5
Input elements in the second matrix :
element - [0],[0] : 12
element - [0],[1] : 43
element - [1],[0] : 23
element - [1],[1] : 2

The First matrix is :

11      43
24      5
The Second matrix is :

12      43
23      2
The Addition of two matrix is :

23      86
47      7
```

Summary

- An array is a collection of one or more values of the same type. Each value is called an element of the array. The elements of the array share the same variable name but each element has its own unique index number (also known as a subscript). An array can be of any type, For example: int, float, char etc.

- 2-D and 3-D arrays are used in more complex data structures, so if you will encounter the needs of manipulation large data sets, these multidimensional arrays can help to do the job.

References

Overiq.com. (2020). *C Programming Tutorial*. https://overiq.com/c-programming-101/

Programiz.com. (n.d.). *Learn C Programming*. https://www.programiz.com/c-programming

Singh, C. (n.d.). *Arrays in C programming with examples*. https://beginnersbook.com/2014/01/c-arrays-example/

MODULE 6

PASSING ARRAYS TO A FUNCTION

Introduction

In C language, arrays can be passed to a function as an argument in a similar way to passing any other variable. The function can then operate on the array elements within its scope. The array name is passed as a pointer, which means that any changes made to the array within the function will be reflected in the original array outside of the function. It is important to note that are passed by reference and not by value, meaning that the function receives a pointer to the original array rather than a copy of the array.

In this module, we are going to discuss and understand how to pass arrays to functions.

Learning Outcomes

At the end of this module, students should be able to:

- Identify and apply on how to pass an array to a Function.

Lesson 1. Passing 1-D Array to a Function (Overiq.com, 2020)

Based in the tutorial “C Programming 101” by Overiq.com (2020), when an array is passed to a function, the changes made by the function affect the original array. When a 1-D array is passed to the function, it is optional to specify the size of the array in the formal arguments. So if we are passing an array of 5 integers then the formal argument of a function can be written in the following two ways.

```
int my_arr[5] = [11,44,66,90,101];
```

1st Way:

```
void function (int a[]) // here the size of the array is omitted
{
    // statements;
}
```

2nd Way:

```
void function(int a[5]) // here the size of the array is
specified
{
    // statements;
}
```

Example:

```
#include<stdio.h>
void new_array(int a[]);

int main()
{
    int my_arr[] = {1,4,9,16,23}, i;
    printf("Original array: \n\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", my_arr[i]);
    }

    my_func(my_arr);

    printf("\n\nModified array : \n\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", my_arr[i]);
    }

    // signal to operating system program ran fine
    return 0;
}

void my_func(int a[5])
{
    int i;

    // increment original elements by 5

    for(i = 0; i < 5; i++)
    {
        a[i] = a[i] + 5;
    }
}
```

Output:

```
Original array:

1 4 9 16 23

Modified array:

6 9 14 21 28
```

How it works?

We know that **my_arr** is a pointer to the first element of the array. So we can pass **my_arr** to the function **my_func()** without using & operator. In line 15, the **my_func()** is called with an actual argument of **my_arr** which is then assigned to **a**. Again note that we are passing the address of **my_arr** to **a**, that means we are using call by reference instead of call by value. So now both **my_arr** and a points to the same array. Inside the function, we are using for loop to increment every element of the array by 5. Since we are operating on the original array all the changes made here effect the original array (Overiq.com, 2020).

Lesson 2. Passing 2-D Array to a Function (Overiq.com, 2020)

Based in the tutorial "C Programming 101" by Overiq.com (2020), Just like a 1-D array, when a 2-D array is passed to a function, the changes made by function effect the original array. But before we discussed this, I want to make a few points clear.

We have learned that in the previous lesson that a Two Dimensional Array in C, when a 2-D is passed to a function it is optional to specify the size of the left most dimensions. So if we have an array of 2 rows and 3 dimensions then it can be passed to a function in the following two ways:

```
int two_d[2][3] = {
                        {99,44,11},
                        {4,66,9}
                    };
```

1st Way:

```
void function(int a[][3])
{
    // statements;
}
```

2nd Way:

```
void function(int a[2][3])
{
    // statements;
}
```

Recall that 2-D arrays are stored in row-major order i.e first row 0 is stored, then next to it row 1 is stored and so on. Therefore, in C, a 2-D array is actually a 1-D array in which each element is itself a 1-D array. Since the name of the array points to the 0th element of the array. In the case of a 2-D array, 0th element is an array.

Example:

```
#include<stdio.h>
void change_twod(int (*a)[3]);

int main()
{
    int i,j, two_d[2][3] = {
                                {99,44,11},
                                {4,66,9}
                            };
    printf("Original array: \n\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("%3d ", two_d[i][j]);
        }
        printf("\n");
    }
    change_twod(two_d);

    printf("\n\nModified array : \n\n");

    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("%3d ", two_d[i][j]);
        }
        printf("\n");
    }
    return 0;
}
void change_twod(int (*arr)[3])
{
    int i, j;
    printf("\n\nIncrementing every element by 5\n");
    // increment original elements by 6
    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            arr[i][j] = arr[i][j] + 5;
        }
    }

}
```

Output:

```
Original array:

99 44 11
4 66 9
Incrementing every element by 5
Modified array :

104 49 16
9 71 14
```

How it works?

As discussed earlier in this section that **two_d** and **arr** are of type pointer to an array of 3 integers. In line 25, **change_twod()** is called with an actual argument of **two_d** which is then assigned to **arr**. Now both **two_d** and **arr** points to the same 2-D array, as a result, changes made inside the function will be visible in the function **main()** (Overiq.com, 2020).

If you're having trouble understanding the lesson, it is highly recommended you watch this video tutorial by Neso Academy in Passing Arrays to a Function - *https://www.youtube.com/watch?v=gNlmJ2WrZSY*

Assessment Task

Coding Problems: Answer the ff. programs based on the given assessment format.

4. Create a C Program that will find repeating elements in a given array.

Sample Output:

```
The given array is :  1  1  2  3  4  5  5
The repeating elements are:  1  5
--------------------------------
```

Summary

- An array is a collection of one or more values of the same type. Each value is called an element of the array. The elements of the array share the same variable name but each element has its own unique index number (also known as a subscript). An array can be of any type, For example: int, float, char etc.

- 2-D and 3-D arrays are used in more complex data structures, so if you will encounter the needs of manipulation large data sets, these multidimensional arrays can help to do the job.

References

Overiq.com. (2020). *C Programming Tutorial*. https://overiq.com/c-programming-101/

MODULE 7

STRINGS

Introduction

C-style strings are really arrays of chars with a little bit of special sauce to indicate where the string ends. In reality, strings are merely just collections of characters sitting next to each other in memory. The only difference is that you cannot modify string literals, whereas you can modify arrays. Functions that take a C-style string will be just as happy to accept string literals unless they modify the string (in which case your program will crash). Some things that might look like strings are not strings; in particular, a character enclosed in single quotes, like this, 'a', is not a string. It's a single character, which can be assigned to a specific location in a string, but which cannot be treated as a string (Allain, 2019).

In this module, we are going to discuss and understand what is strings and the different library functions used to manipulate strings.

Learning Outcomes

At the end of this module, students should be able to:

- Discuss and Explain what is string literals, how it is used and stored.
- Apply the common library functions used in manipulating strings.

Lesson 1. What is Strings? (Overiq.com, 2020)

The programs we have written so far deals only with numbers or characters, but a real-world program should be able to store and manipulate text when needed. Unfortunately, C offers no separate data type for strings, Languages like Java and C# provides a separate type for strings but this is not the case with C.

Based in the tutorial “C Programming 101” by Overiq.com (2020), In C strings are stored as an array of characters terminated by a null character. An array of characters is a string only if it's the last element is a null character ('\0'). The null character is an escape sequence just like \n (newline), \t (tab) with ASCII value of 0. **For example:**

```
char name[10] = {'s', 't', 'r', 'i', 'n', 'g' ,'\0'};
```

So, we can say that a string is just a one-dimensional array of characters with a null character ('\0') as it's the last element.

Lesson 2. String Literal (Overiq.com, 2020)

Based in the tutorial “C Programming 101” by Overiq.com (2020), a string literal is just a sequence of characters enclosed in double quotes (" "). It is also known as a string constant.

Here are some examples of string literals:

```
"I am learning C"
"My Lucky Number is 1"
"Hello World!"
""
```

The double quotes (" ") are not part of a string literal they are just used to delineate (i.e mark boundaries) a string. Whenever you create a string literal in a program the compiler automatically adds null character ('\0') at the end.

How string literals are stored?

As discussed, a string is actually an array of characters terminated by a null character ('\0'). Whenever compiler sees string literal of length **n** it allocates **n + 1** consecutive bytes of memory for the string. This memory will contain all the characters inside a string, plus null character ('\0') at the end of the string. So, the string literal "Hello World" will be stored in the memory as:

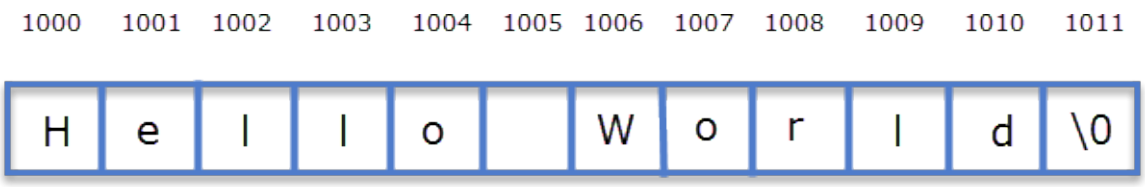

Figure 7.11. String Literals (Overiq.com, 2020)

As you can see string literal "Hello World" is stored as an array of 12 characters (including '\0').

A string literal can also be empty.

The "" (empty string literal, contains only '\0'). It will be stored in the memory as an array of 1 character.

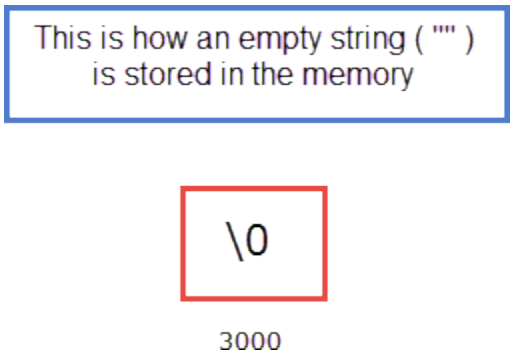

Figure 7.12. Empty String stored in a Memory (Overiq.com, 2020)

Lesson 3. printf() and scanf() revisited (Overiq.com, 2020)

Based in the tutorial “C Programming 101” by Overiq.com (2020), If you look at the prototype of scanf() and print(), you will find that both function expects a value of type (char*) as their first argument.

```
int printf (const char*, ...);
int scanf (const char*, ...);
```

Well now you know when you call **printf()** function as:

```
printf("Hello World");
```

You are actually passing an address of "Hello World" i.e a pointer to the first letter of the array which is '**H**'.

Now, we are done how strings literals works and stored. We're going to proceed to the next lesson about strings.

Lesson 4. C Programming Strings (Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), In C programming, a string is a sequence of characters terminated with a null character \0. For example:

```
char c[] = "c string";
```

When the compiler encounters a sequence of characters enclosed in the double quotation marks, it appends a null character \0 at the end by default.

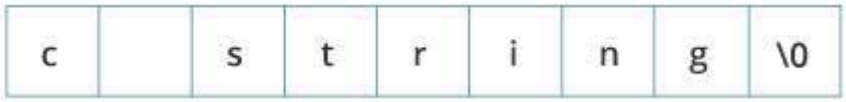

Figure 7.13 String Sequence (Programiz.com, n.d.)

How to declare a String?

Here's how you can declare strings:

```
char s[5];
```

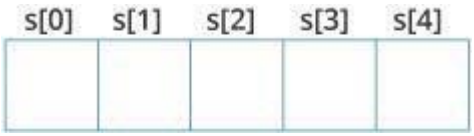

Figure 7.14 String Declaration (Programiz.com, n.d.)

Here, we have declared a string of 5 characters. As you can see, you can access a specific character in a string using indices just like an array.

Assigning Values to Strings

Arrays and strings are second-class citizens in C; they do not support the assignment operator once it is declared (Programiz.com, n.d.). For example,

```
char c[100];
c = "C programming";  // Error! array type is not assignable.
```

Just like any variable, you can value to string by using this format. The value **c [100]**, it means you declare a variable named **c** that can store 100 characters.

Read String from the User

You can use the **scanf()** function to read a string.

The **scanf()** function reads the sequence of characters until it encounters whitespace (space, newline, tab, etc.).

Example 1: scanf() to read a string

```
#include <stdio.h>
int main()
{
    char name[20];
    printf("Enter name: ");
    scanf("%s", name);
    printf("Your name is %s.", name);
    return 0;
}
```

Output:

```
Enter name: Ren Santos
Your name is Ren.
```

Even though **Ren Santos** was entered in the above program, only "**Ren**" was stored in the name string. It's because there was a space after **Ren**.

How to read a line of text?

You can use the **fgets()** function to read a line of string. And, you can use **puts()** to display the string (Programiz.com, n.d.).

Example 2: fgets() and puts()

```
#include <stdio.h>
int main()
{
    char name[30];
    printf("Enter name: ");
    fgets(name, sizeof(name), stdin);  // read string
    printf("Name: ");
    puts(name);    // display string
    return 0;
}
```

Output:

```
Enter name: Ren Santos
Name: Ren Santos
```

Here, we have used **fgets()** function to read a string from the user.

```
fgets(name, sizeof(name), stdlin); // read string
```

The **sizeof(name)** results to 30. Hence, we can take a maximum of 30 characters as input which is the size of the **name** string.

To print the string, we have used **puts(name)**;

Note: The **gets()** function can also be to take input from the user. However, it is removed from the C standard. You can still use it.

It's because **gets()** allows you to input any length of characters. Hence, there might be a buffer overflow.

Lesson 5. String Manipulation using Library Functions
(Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), you need to often manipulate strings according to the need of a problem. Most, if not all, of the time string manipulation can be done manually but, this makes programming complex and large.

To solve this, C supports a large number of string handling functions in the standard library "**string.h**".

Few commonly used strings handling functions are discussed below:

Table 7.2 String Handling Function (Programiz.com, n.d.)

Function	Description
strlen()	computes string's length
strcpy()	copies a string to another
strcat()	concatenate or joins two strings
strcmp()	compares two strings
strlwr()	converts string to lowercase
strupr()	converts string to uppercase

Strings handling functions are defined under "**string.h**" header file.

```
#include <string.h>
```

Lesson 5.1 C strlen() Function (Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), the **strlen()** function calculates the length of a given string.

The **strlen()** function takes a string as an argument and returns its length. The returned value is of type **size_t** (the unsigned integer type).

It is defined in the **<string.h>** header file.

Example: C strlen() Function

```
#include <stdio.h>
#include <string.h>
int main()
{
    char a[20]="Program";
    char b[20]={'P','r','o','g','r','a','m','\0'};

    // using the %zu format specifier to print size_t
    printf("Length of string a = %zu \n",strlen(a));
    printf("Length of string b = %zu \n",strlen(b));

    return 0;
}
```

Output:

```
Length of string a = 7
Length of string b = 7
```

Note: The **strlen()** function doesn't count the null character \0 while calculating the length.

Lesson 5.2 C strcpy() Function (Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), The function prototype of **strcpy()** is:

```
char* strcpy(char* destination, const char* source);
```

- The **strcpy()** function copies the string pointed by source (including the null character) to the destination.
- The **strcpy()** function also returns the copied string.

The **strcpy()** function is defined in the **string.h** header file.

Example: strcpy()

```
#include <stdio.h>
#include <string.h>

int main() {
   char str1[20] = "C programming";
   char str2[20];

   // copying str1 to str2
   strcpy(str2, str1);

   puts(str2); // C programming

   return 0;
}
```

Output:

```
C programming
```

Note: When you use **strcpy()**, the size of the destination string should be large enough to store the copied string. Otherwise, it may result in undefined behavior.

Lesson 5.3 C strcat() Function (Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), In C programming, the **strcat()** function concatenates (joins) two strings.

The function definition of **strcat()** is:

```
char *strcat(char *destination, const char *source)
```

It is defined in the **string.h** header file.

C strcat() Arguments

As you can see, the **strcat()** function takes two arguments:

- **Destination** – destination string
- **Source** – source string

The **strcat()** function concatenates the **destination** string and the **source** string, and the result is stored in the **destination** string.

Example: C strcat() Function

```
#include <stdio.h>
#include <string.h>
int main() {
   char str1[100] = "This is ", str2[] = "programiz.com";

   // concatenates str1 and str2
   // the resultant string is stored in str1.
   strcat(str1, str2);

   puts(str1);
   puts(str2);

   return 0;
}
```

Output:

```
This is programiz.com
programiz.com
```

Note: When we use **strcat(),** the size of the destination string should be large enough to store the resultant string. If not, we will get the segmentation fault error.

Lesson 5.4 C strcmp() Function (Programiz.com, n.d.)

Based in the C Tutorial by Programiz.com (n.d), The **strcmp()** function compares two strings and returns **0** if both strings are identical.

C strcmp() Prototype

```
int strcmp (const char* str1, const char* str2);
```

It is defined in the **string.h** header file.

The **strcmp()** function takes two strings and returns an integer.

The **strcmp()** compares two strings character by character. If the first character of two strings is equal, the next character of two strings is compared. This continues until the corresponding characters of two strings are different or a null character '\0' is reached.

Table 7.3 Return Value of strcmp() (Programiz.com, n.d.)

Return Value	Remarks
0	if both strings are identical (equal)
Negative Integer	if the ASCII value of the first unmatched character is less than the second.
Positive Integer	if the ASCII value of the first unmatched character is greater than the second.

Example: C strcmp() Function

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "abcd", str2[] = "abCd", str3[] = "abcd";
    int result;

    // comparing strings str1 and str2
    result = strcmp(str1, str2);
    printf("strcmp(str1, str2) = %d\n", result);

    // comparing strings str1 and str3
    result = strcmp(str1, str3);
    printf("strcmp(str1, str3) = %d\n", result);

    return 0;
}
```

Output:

```
strcmp(str1, str2) = 32
strcmp(str1, str3) = 0
```

The first unmatched character between string **str1** and **str2** is third character. The ASCII value of 'c' is 99 and the ASCII value of 'C' is 67. Hence, when strings **str1** and **str2** are compared, the return value is 32.

When strings **str1** and **str3** are compared, the result is 0 because both strings are identical.

Lesson 5.5 C strlwr() Function (Geeksforgeeks.org, 2018)

Based in the tutorial “strlwr() Function in C” by geeksforgeeks.org (2018), the **strlwr( )** function is a built-in function in C and is used to convert a given string into lowercase.

C strlwr() Prototype

```
char *strlwr(char *str);
```

Example 1:

```
#include<stdio.h>
#include<string.h>

int main()
{
    char str[ ] = "C PROGRAMMING";

    // converting the given string into lowercase.
    printf("%s\n",strlwr (str));

    return  0;
}
```

Output:

```
C programming
```

Example 2:

```
#include<stdio.h>
#include <string.h>

int main()
{
  char str[] = "C programming TutorIaL";

  printf("Given string is: %s\n",str);

  printf("\nString after converting to the "
             "lowercase is: %s",strlwr(str));

  return 0;
}
```

Output:

```
Given string is: C programming TutorIaL

String after converting to the lowercase is: c programming
tutorial
```

Lesson 5.6 C strupr() Function (Geeksforgeeks.org, 2018)

Based in the tutorial “strupr() Function in C” by geeksforgeeks.org (2018), the **strupr(**) function is used to converts a given string to uppercase.

C strlwr() Prototype

```
char *strupr(char *str);
```

Example 1:

```
#include<stdio.h>
#include<string.h>

int main()
{
    char str[ ] = "c programming";
    //converting the given string into uppercase.
    printf("%s\n", strupr (str));
    return  0;
}
```

Output:

```
C PROGRAMMING
```

Example 2:

```
#include<stdio.h>
#include <string.h>

int main()
{
  char str[] = "C ProgRamMing TuTORial";

  printf("Given string is: %s\n", str);

  printf("\nstring after converting to the uppercase is: %s",
strupr(str));
  return 0;
}
```

Output:

```
Given string is: C ProgRamMing TuTORial

string after converting to the uppercase is: C PROGRAMMING
TUTORIAL
```

For more information about the other library functions used in manipulation strings, you can visit this link: https://fresh2refresh.com/c-programming/c-function/string-h-library-functions/

You can watch this video tutorial by Neso Academy in using C Strings:

- *Basics of String Literals - https://www.youtube.com/watch?v=llqiTmcK1Eg&list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW*
- *Storing the String Literals - https://www.youtube.com/watch?v=zNCgj3mfixw&list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW&index=2*
- *Declaring and Initializing String Variables - https://www.youtube.com/watch?v=cnfRyvo41Bs&list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW&index=5*
- *Writing Strings using printf() and puts Functions - https://www.youtube.com/watch?v=wW7u_WrkY6Q&list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW&index=6*
- *Reading Strings using scanf() and gets Functions - https://www.youtube.com/watch?v=2HasEQe5VR0&list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW&index=7*

Assessment Task

Coding Problems: Answer the ff. programs based on the given assessment format.

5. Create a C Program that will print the individual characters of a string in a reverse order.

Sample Output:

```
Input the string : C PROGRAMMING
The characters of the string in reverse are :

  G  N  I  M  M  A  R  G  O  R  P     C
```

6. Create a C Program that will count the total number of letters, digits and special characters in a string.

Sample Output:

```
Input the string : Link - www.ilearnu.edu.ph/student_1
Number of Alphabets in the string is : 26
Number of Digits in the string is : 1
Number of Special characters in the string is : 9
```

7. Create a C Program that can read a sentence and replace the lowercase letters to uppercase letters and do it vice-versa.

Sample Output:

```
Input the string : C pRoGramMinG

The given sentence is    : C pRoGramMinG
After Case changed the string  is: c PrOgRAMmINg
```

Summary

- String is a data type used in programming, such as an integer and floating-point unit, but is used to represent text rather than numbers. It is comprised of a set of characters that can also contain spaces and numbers. You can also consider a string as a one-dimensional array because you can access a specific character in a string using indices.

- The commonly used library functions in manipulation strings in C are strlen(), strcpy(), strcmp(), strcat(), strlwr() and strupr().

References

Allain, A. (2019). *Recursion in C*. https://www.cprogramming.com/tutorial/c/lesson16.html

Geeksforgeeks.org. (2022). *C Programming Language*. https://www.geeksforgeeks.org/c-programming-language/

Overiq.com. (2020). *C Programming Tutorial*. https://overiq.com/c-programming-101/

Programiz.com. (n.d.). *Learn C Programming*. https://www.programiz.com/c-programming

- **END OF THE MIDTERM MODULE** -

CHECK YOUR EXAM SCHEDULE FOR THIS COURSE.

DO NOT FORGET TO TAKE THE EXAM AS SCHEDULED.

THANK YOU AND GOD BLESS